

DATA 266 - Homework 3 Findings

Prompt Engineering and Self-Attention with Causal Masking

Personal Parameters

Parameter	Value
SID4	9486
SEED	9486
SLICE	486
HP_ID	0
CLS_A	6
CLS_B	0
HP_ID arm	Capacity-low

I used SEED = 9486 for Python, NumPy, and PyTorch reproducibility. The other personal parameters are reported as required by the standing Assignment 1 instructions; this homework does not use SLICE or the class parameters for dataset selection.

Introduction

This homework has two parts. First, I tested six prompt-engineering techniques using two coded examples per technique. Second, I implemented single-head scaled dot-product self-attention from scratch in PyTorch on the required text and compared unmasked attention with causal-masked attention. The results in this report come from the executed HW3.ipynb notebook.

Submission contents

Part	What was completed
Part 1	Six techniques, 12 coded prompt examples, and an output comparison.
Part 2	Trainable embeddings and Q/K/V projections, next-token training, attention heatmaps, and causal-mask checks.
Verification	Attention shape, softmax row sums, lower-triangular mask, and zero future attention were checked directly on tensors.

Part 1: Prompt Engineering

Each example was called through code using the professor's LangChain and ChatGoogleGenerativeAI pattern. There were no screenshots or hand-written chat transcripts. The examples below summarize the actual prompts and saved responses.

Examples 1-6

Technique	Example 1 and result	Example 2 and result
Zero-Shot	Math: $17 \times 6 \rightarrow$ 102 .	Sentiment classification of a six-hour delayed flight $\rightarrow$ Negative .
Few-Shot	Even-number sum example $\rightarrow$ False , following the demonstrated label format.	Technology classification example $\rightarrow$ Technology .
Chain-of-Thought	Even-number sum $\rightarrow$ explained $46 + 8 + 6 + 4 = 64$, so False .	Robin/feathers logic $\rightarrow$ explained the rule and concluded Yes .
Zero-Shot CoT	Cupcake updates: $94 - 8 + 6 - 4 =$ 88 .	Zorbs $\rightarrow$ Flurps $\rightarrow$ Wibbles, so Yes by transitivity.

How the techniques differ

Technique	Prompt structure	Observed purpose
Zero-Shot	One instruction with no examples.	Direct answers; simple and quick.
Few-Shot	Examples show the desired input/output pattern.	Helps the model infer labels and format.
Chain-of-Thought	Examples include intermediate reasoning.	Encourages an explicit reasoning trace.
Zero-Shot CoT	Adds 'Let's think step by step' without examples.	Requests reasoning without demonstrations.

Part 1: Remaining Prompt Results

Examples 7-12

Technique	Example 1 and result	Example 2 and result
Meta-Prompting	Asked for a general strategy for deciding whether an even-number sum is odd or even. No visible final answer was returned.	Asked for a sentiment-classification checklist. The response began a useful checklist but stopped mid-sentence.
Tree of Thoughts	Orange problem: two different operation orders both give 12 , although the saved response stopped before a final total line.	Sheep riddle: compared $17 - 9 = 8$ with the idiomatic reading and correctly selected 9 .

Comparison and findings

Zero-shot worked well for direct questions. Few-shot examples helped specify the expected labels and format. Chain-of-thought and zero-shot chain-of-thought produced visible intermediate reasoning for the arithmetic and logic examples. Tree of Thoughts explicitly compared multiple paths, which was useful for the sheep wording. Meta-prompting produced a strategy or checklist instead of directly solving one instance, but both saved responses were incomplete in this run.

Overall, the outputs do not justify ranking one technique as universally better. The results show different behaviors and also show that response completeness can depend on the model and output budget. The incomplete responses were retained as observed outputs rather than replaced with invented text.

Prompting summary

Technique	Main strength	Main limitation
Zero-Shot	Fast and simple for familiar tasks.	Provides little guidance about reasoning or format.
Few-Shot	Demonstrates the desired pattern.	Requires carefully chosen examples.
Chain-of-Thought / Zero-Shot CoT	Makes reasoning steps more visible.	Longer responses and possible reasoning errors.
Meta-Prompting	Creates reusable strategies or checklists.	Requires a follow-up task to apply the strategy.
Tree of Thoughts	Compares multiple paths or interpretations.	Longest and most structured approach.

Part 2: Self-Attention from Scratch

Dataset and tokenization

I used the exact text required by the assignment. Word-level tokenization lowercased the text and kept letter runs while dropping punctuation. This produced 46 tokens and 39 unique vocabulary words. The next-token task uses every token except the last as input and the same sequence shifted one position to the right as the target.

Required dataset text

Neural networks are powerful models for learning representations from data. They consist of layers of interconnected neurons. Attention mechanisms allow models to focus on relevant parts of the input. Transformers rely entirely on attention instead of recurrence. Autoregressive models generate text one token at a time.

Model and training setup

Setting	Value
Architecture	SingleHeadSelfAttentionLM: nn.Embedding + Wq/Wk/Wv + Wo + lm_head
Attention type	Single-head scaled dot-product attention; no bias in the projection layers
Dimension	d_model = 64 and d_k = 64
Objective	Cross-entropy loss for next-token prediction
Optimizer	Adam; learning rate = 0.01; 600 epochs
Training mask	Causal mask applied during training
Device and seed	CPU; SEED = 9486 immediately before model creation

Training result

The logged loss values were 0.0407 at epoch 100, 0.1666 at epoch 200, 0.1657 at epoch 300, 0.1641 at epoch 400, 0.4846 at epoch 500, and 0.5276 at epoch 600. The loss decreased early but became unstable later. Therefore, the report separates the verification of the attention mechanics from claims about clean language-model convergence.

The implementation used only basic PyTorch operations and layers. It did not use nn.MultiheadAttention, nn.Transformer, HuggingFace transformer classes, positional encoding, or multi-head attention.

Attention Calculation and Masking

Scaled dot-product attention

The attention computation was implemented directly using the following steps:

```
scores = Q @ K.T / sqrt(d_k)
attention_weights = softmax(scores)
output = attention_weights @ V
```

The full sequence was passed through the trained model to obtain the post-training attention weights. Without a mask, each token can attend to positions before or after it. For causal attention, the score matrix is masked before softmax using a lower-triangular boolean matrix. Entries above the diagonal are assigned a very negative value so their softmax weights become zero.

Causal mask

For position i , the mask allows positions $j \leq i$ and blocks positions $j > i$. A five-position example from the notebook was:

```
[[ True, False, False, False, False],
 [ True, True, False, False, False],
 [ True, True, True, False, False],
 [ True, True, True, True, False],
 [ True, True, True, True, True]]
```

Verification results

Check	Result
Attention matrix shape	Unmasked and masked: (1, 46, 46)
Softmax row sums	Minimum = 0.9999998808; maximum = 1.0000002384
Mask structure	Passed exact equality check against torch.tril(...)
Total masked mass above diagonal	0.0
Maximum future-position weight	0.0
All future positions blocked	Passed

Interpretation

The checks confirm that the attention rows are normalized and that the causal mask blocks 100% of future-token attention. The unmasked and masked matrices use the same trained embeddings and Q/K/V weights in the final inference comparison; only the inference mask changes. The training loss was not consistently decreasing, so the strongest conclusion is about correct attention and masking behavior rather than model quality.

Unmasked Self-Attention

This heatmap was generated after training on the full 46-token sequence without a causal mask. Bright cells can occur both below and above the diagonal because a token is allowed to attend to earlier and later positions.

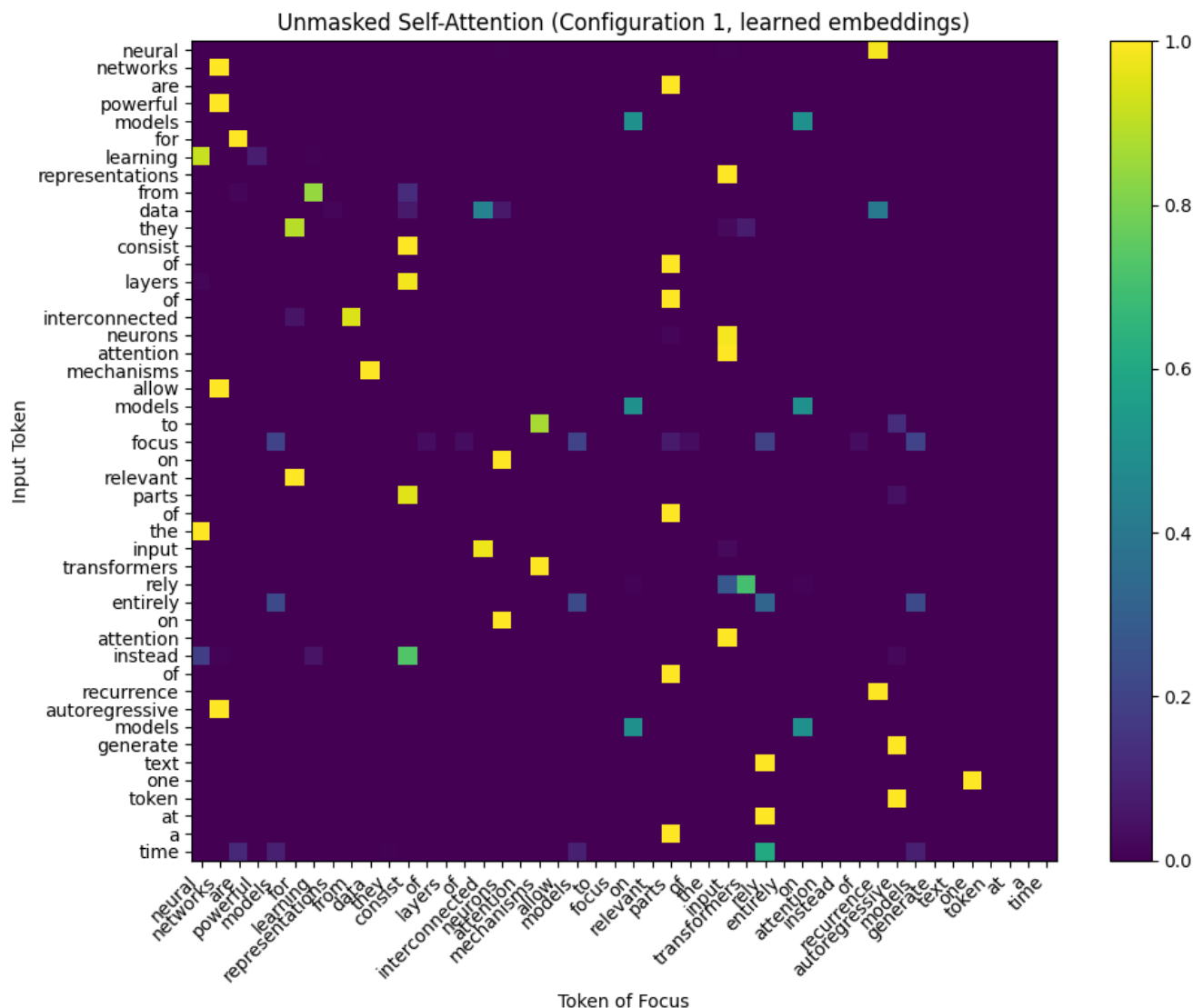


Figure 1. Unmasked self-attention after training. The axes use the full word-token sequence.

The matrix has shape (1, 46, 46) before selecting the sequence matrix for visualization. Every row sums to approximately one because the weights are produced by softmax.

Causal-Masked Self-Attention and Conclusion

The following heatmap applies the lower-triangular causal mask before softmax. The complete upper-triangular region is zero, so no token attends to a future position.

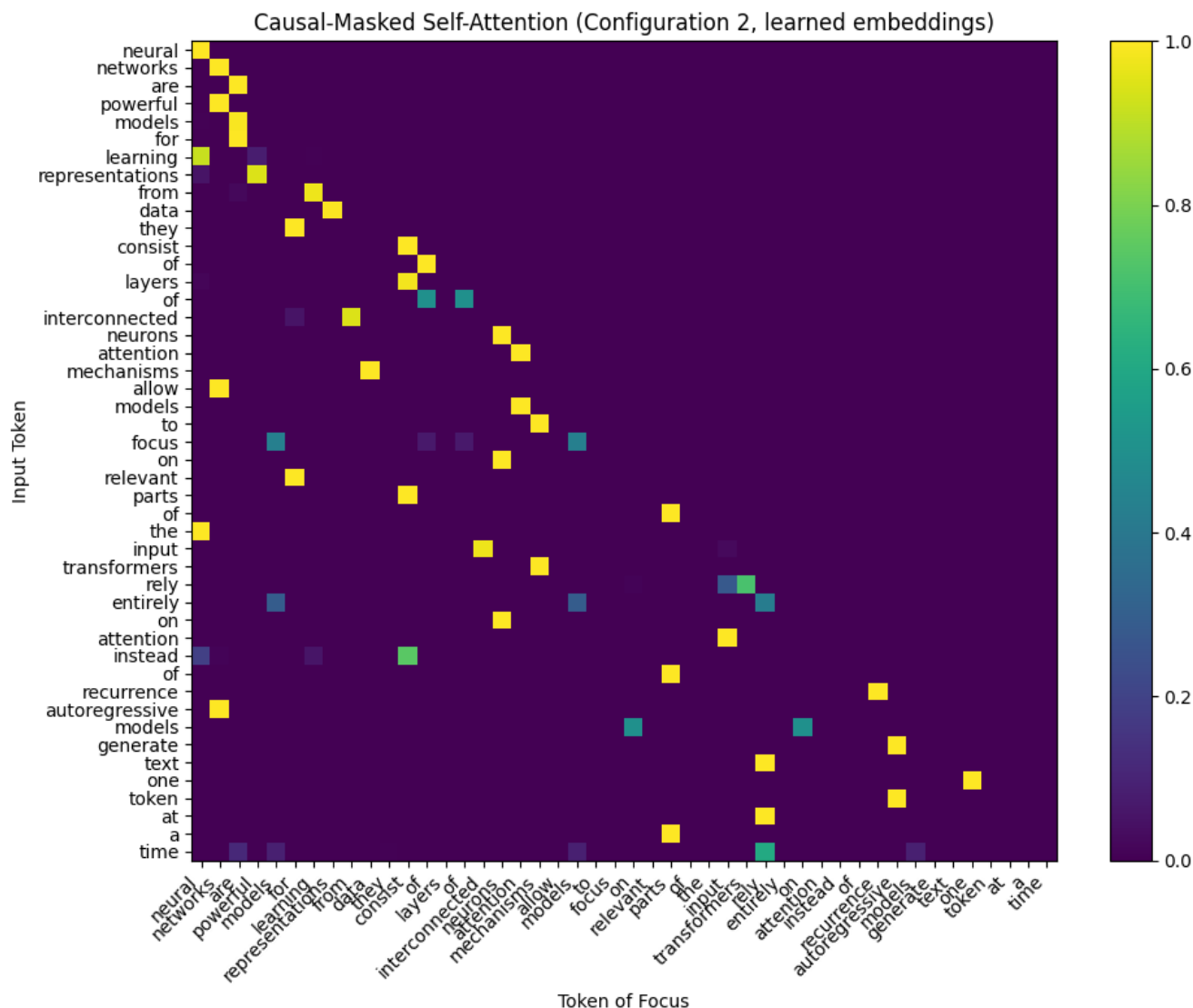


Figure 2. Causal-masked self-attention after training. Future-token attention is zero.

Conclusion

In Part 1, the six prompting techniques produced different response structures. Direct prompting was concise, few-shot prompting demonstrated a pattern, reasoning prompts exposed intermediate steps, meta-prompting requested reusable strategies, and Tree of Thoughts compared alternatives. Three of the 12 saved responses were incomplete, so the results do not support a universal ranking of techniques.

In Part 2, the from-scratch implementation produced the required full-sequence attention matrices and visualizations. The shape and softmax checks passed, the causal mask was exactly lower triangular, and all future-position weights were zero. These results verify the key attention and masking behavior even though the particular training run became unstable after its early loss decrease.